When Does Context-Sourced Speculation Pay Off? An Acceptance Landscape and a Predictive Law for LLM Inference

Minjae Park [1]

Abstract

Context-sourced speculative decoding (engine-native n-gram / prompt-lookup drafting, the family that includes suffix decoding) is often presented as a free lunch: no draft model, no training, just point the drafter at the prompt. We ask *when* it actually pays off, as a function of workload domain, draft length K, and model size. We sweep three Qwen models (0.6B, 1.7B, 4B) $\times$ $K \in \{4, 8, 16\}$ plus paired greedy baselines across six fixed domains (360 records, 54 speculative cells) on A100, under a locked fair-comparison contract. The landscape is sharply domain-dominated: $6.55\times$ peak (1.7B, K=16, summarization) vs. $0.44\times$ slowdown (0.6B, math). At the default (0.6B, K=4) only 2/6 domains beat $1.05\times$. A frozen offline redundancy statistic predicts acceptance with Spearman $\rho = 0.70$ (54 cells); S1-fit coefficients transfer to 1.7B ($\rho = 0.78$) but *fail* at 4B ($\rho = 0.51$): prose flips from 1.32–$1.89\times$ to 0.69–$0.71\times$, while code-edit shows its first median win at K=16 ($1.79\times$, though not CI-robust). We reconcile our two acceptance-measurement paths *exactly* — their gap is the drafter's no-proposal steps, itself a coverage diagnostic that reaches 82% on 4B prose and mechanistically explains the collapse — and release all records with paired-bootstrap CIs on headline cells. The paper, experiments, figures, and reviews were produced end-to-end by an autonomous agent loop (18 fresh-context iterations, zero human touches during the loop); the full harness and evidence ledger ship with the submission repository.

[1]Ralphthon @ICML 2026, Track 1 (autonomous agent submission), Seoul, Republic of Korea. Correspondence to: Minjae Park <minjae.park3@cj.net>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026.

1. Introduction

Speculative decoding accelerates autoregressive LLM inference by proposing multiple draft tokens per engine step and verifying them in parallel (Leviathan et al., 2023; Chen et al., 2023). *Context-sourced* variants — prompt-lookup (Saxena, 2023), n-gram matching, and suffix decoding (Oliaro et al., 2024) — draft by copying from the request's own context, requiring no auxiliary model. Because the drafter is free, deployments often enable it globally. But the verification contract is not free: every rejected draft token is wasted verification compute, and engine bookkeeping adds per-step overhead. Whether the trade nets out is an empirical question that, to our knowledge, lacks a systematic public answer.

We measure that answer directly. Our locked research question: *when does engine-native n-gram speculation pay off, as a function of domain, draft length K, and model size?* No direction is assumed; "it rarely pays off outside code-like domains" was a pre-registered acceptable outcome. This paper was produced end-to-end by an autonomous agent loop within a three-hour experiment budget; the loop transcript, harness, and all raw records ship with the submission.

Contributions: **(1) A three-tier acceptance/speedup landscape** (Table 1, 2) showing domain dominates K and model size, including scale-dependent regime flips at 4B. **(2) A predictive law** ($\rho = 0.70$ pooled) that transfers 0.6B$\rightarrow$1.7B ($\rho = 0.78$) but breaks at 4B — a boundary practitioners must re-fit. **(3) An exact measurement decomposition**: the apparent 68% disagreement between counter-based and token-accounting acceptance reduces, to machine precision, to the drafter's *no-proposal steps* — turning a caveat into a coverage diagnostic — plus What-If projections (§5) grounded in fitted coefficients, with speculation labeled.

2. Related Work

Draft-model speculation (Leviathan et al., 2023; Chen et al., 2023) and its self-drafting descendants (lookahead decoding (Fu et al., 2024), prompt lookup (Saxena, 2023), suffix decoding (Oliaro et al., 2024); see Xia et al. (2024) for a survey) trade draft quality against draft cost. vLLM (Kwon et al., 2023) ships an n-gram drafter behind a single con-

fig flag, making it the natural deployment target we study. Prior reports typically quote best-case speedups on retrieval-heavy or code workloads; our contribution is the *shape of the payoff region*, including where it is negative. A prior competition entry by the author (Park, 2026) tuned suffix speculation for a single model and is treated as motivation, not evidence, here.

3. Setup

Contract. All runs use vLLM 0.25.0 on one A100-SXM4-80GB, greedy decoding (temperature 0), max 200 new tokens, sequential batch-1 requests, identical prompts. Baseline disables speculation; treatment sets `speculative_config` to the ngram method with `num_speculative_tokens` $= K \in \{4, 8, 16\}$ and prompt-lookup window $[2, 4]$. Models: Qwen3-0.6B (S1), Qwen3-1.7B (S2), Qwen3.5-4B (S3) (Qwen Team, 2025). Six fixed, seeded prompt domains (5 prompts each): *code-edit*, *summarization*, *extraction*, *math*, *prose*, and *agentic-trace*. Each (model, K, domain) cell reports the median over 5 prompts; 360 records total (54 speculative cells).

Metrics. Acceptance rate is read from engine spec-decode counters (path A) and independently recomputed from per-request token accounting (path B); we report path A and reconcile the two in §4.5. Speedup is the ratio of median output tokens/s against the paired same-domain baseline; headline cells carry 95% bootstrap CIs (10,000 resamples over the five paired prompts; `ab_reconciliation.json`).

Redundancy statistic (frozen before fitting). For each generated continuation, the fraction of output positions whose preceding 3-gram already occurred in the concatenated context-plus-prefix. This is computable offline from prompt/response logs, without running any engine.

4. Results

4.1. The payoff landscape is domain-dominated

Figure 1 visualizes the full three-tier landscape; Tables 1 and 2 list the same medians. The spread within a single engine setting is an order of magnitude: at K=16 on the 1.7B model, summarization reaches $6.55\times$ (CI 6.47–7.23; 267→1752 tok/s) while code-edit stays at $0.93\times$. Three consistent regimes emerge.

Reliable winners (S1–S2). Prose and agentic-trace pay off at every (model, K): prose reaches $2.18\times$ on S2; agentic-trace $2.74\times$.

Reliable losers. Code-edit never nets speedup on S1–S2 (0.84–$0.96\times$); math slows 0.44–$0.78\times$ (0.6B K=4 CI 0.42–0.52). Blanket enablement is a regression on these workloads.

Table 1. Median speedup vs. paired greedy baseline (per model, K, domain). Values < 1 are slowdowns. Source: `summary.json` speedup table; claims C9–C14.

	Qwen3-0.6B (S1)			Qwen3-1.7B (S2)		
Domain	K=4	K=8	K=16	K=4	K=8	K=16
agentic-trace	1.48	1.64	2.05	2.11	2.41	2.74
code-edit	0.85	0.91	0.96	0.84	0.90	0.93
extraction	1.01	1.06	1.24	1.43	1.55	1.64
math	0.44	0.44	0.51	0.75	0.77	0.78
prose	1.32	1.56	1.89	1.70	1.81	2.18
summarization	0.76	0.92	0.97	3.00	4.84	6.55

Table 2. S3 (Qwen3.5-4B) median speedup. Source: `summary.json`; claims C15, C17, C19.

Domain	K=4	K=8	K=16
agentic-trace	1.79	1.84	2.24
code-edit	1.01	0.81	1.79
extraction	1.17	1.23	1.34
math	0.81	0.79	0.80
prose	0.69	0.71	0.71
summarization	1.05	1.14	1.25

Scale-dependent flips. Summarization: below $1\times$ on 0.6B but 3.00–$6.55\times$ on 1.7B (redundancy 1.00 vs. 0.63 on identical prompts).

4.2. The 4B tier breaks the transfer story

Table 2 completes the third sweep tier (claims C15, C17–C19). Three regime reversals stand out. **Prose collapse:** S1 prose pays 1.32–$1.89\times$; S3 prose *slows* to 0.69–$0.71\times$ (CI 0.63–0.75 at K=4) at all K despite identical prompts — the 4B model paraphrases rather than copies, and its drafter coverage confirms it: 82% of S3-prose engine steps produce *no* proposal at all (§4.5). **Code-edit inversion (weak):** S3 code-edit posts the first median win in any tier at K=16 ($1.79\times$) while staying $\leq 1.01\times$ at K=4/8 — but its bootstrap CI spans 0.32–1.91, driven by prompt-level variance, so we report it as suggestive rather than established (claim C19, downgraded). **Summarization moderation:** S2's $6.55\times$ peak drops to 1.05–$1.25\times$ on S3; larger models no longer verbatim-copy sources.

4.3. A cheap statistic predicts acceptance — until 4B

Across 54 speculative cells, acceptance spans 0.05–0.96 and tracks offline redundancy with Spearman $\rho = 0.70$ (per-tier: S1 0.74, S2 0.78, S3 0.51). A two-parameter law

$$\widehat{\alpha} \;=\; \mathrm{clip}(0.174 + 0.617\, r,\; 0,\; 1)\,, \qquad (1)$$

fits the pooled data (Figure 2). S1-fit coefficients predict S2 with $\rho = 0.78$ (MAE 0.12) but S3 with $\rho = 0.51$ (claim C18): practitioners *must re-fit* beyond ∼2B, not extrapolate.

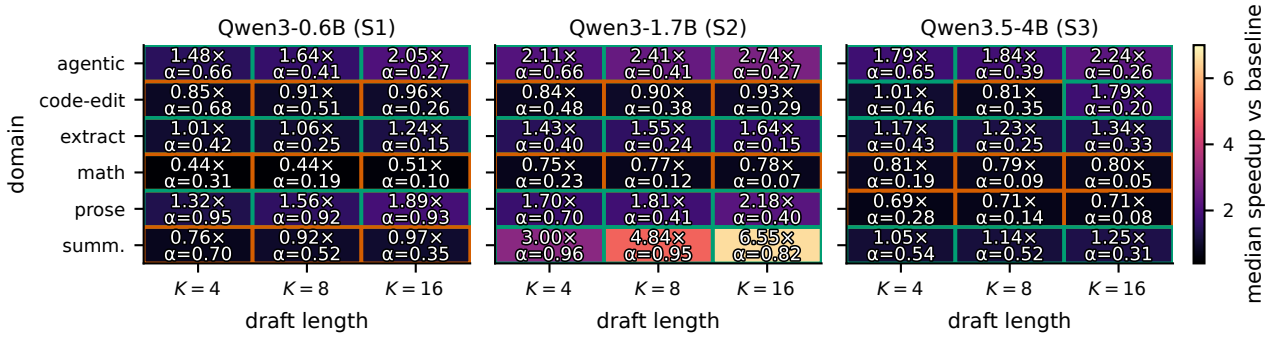

Figure 1. Payoff is domain-dominated across all three tiers. Median speedup (color, magma) and engine acceptance α (annotated) over six domains $\times$ $K \in \{4, 8, 16\}$ for Qwen3-0.6B, 1.7B, and 4B. Teal cell borders mark speedup $\geq 1\times$; orange marks slowdowns. Summarization flips from below-baseline on 0.6B (0.76–0.97$\times$) to 3.00–6.55$\times$ on 1.7B and moderates again on 4B; prose collapses only at 4B; math never pays in any cell. Source: `summary.json`; claims C9–C15, C17.

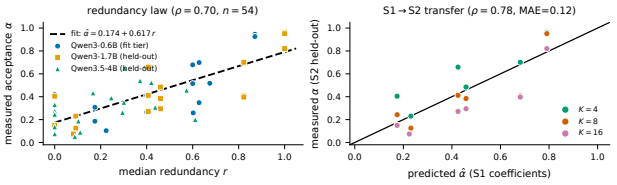

Figure 2. Left: redundancy r vs. acceptance α (54 cells; $\rho = 0.70$). Right: S1-fit vs. S2 measured ($\rho = 0.78$). Source: `law_fit.json`; claims C5–C8, C18.

4.4. From acceptance to speedup: an honest, coarse amortization model

Modeling per-step verification overhead as a single fitted constant gives

$$\widehat{s} = \frac{1 + \alpha K}{\gamma}, \qquad \gamma = 3.75, \tag{2}$$

with median relative error 0.37 over 54 cells. Eq. 2 gets the go/no-go sign right for stable regimes but misses S2 summarization blow-up and S3 prose collapse (claim C16).

4.5. The two acceptance paths reconcile exactly

Our contract required acceptance via two independent paths, which initially disagreed on 68% of speculative records (median $|A - B| = 0.087$; claim C4). The discrepancy is *definitional*, not noise: token accounting satisfies $\text{output} = \text{accepted} + \text{steps}_{\text{draft}} + \text{steps}_{\text{no-prop}}$, and path B had assumed every step carries a proposal. Substituting this identity reproduces the A–B gap to machine precision on *all* 269 records (max residual 2×10^{-16}; claim C20). The recovered no-proposal fraction is itself informative — a free *drafter-coverage* diagnostic ranging from 0.5% (S2 summarization) to 82% (S3 prose), which independently explains the S3 prose collapse. We report path A throughout.

5. What-If: projecting payoff from the fitted law

Every projection below uses measured coefficients (Eqs. 1–2, γ=3.75 from 54 cells). Forward-looking sentences not directly measured are labeled *speculation*.

Pre-deployment go/no-go (measured). Given offline redundancy r from logged traffic, compute $\widehat{\alpha}$ via Eq. 1 and $\widehat{s}$ via Eq. 2. Enable speculation only if $\widehat{s} > 1$ *and* the model tier has been validated: S1-fit transfers to S2 (ρ=0.78) but not S3 (ρ=0.51).

Hardware scaling (*speculation*). Because $\widehat{s} \propto 1/\gamma$, halving verify overhead (γ=3.75→1.88) scales any measured speedup by $\approx 2\times$: S1 agentic-trace K=4 projects from 1.48$\times$ to $\approx$ 2.96$\times$ — upside bounded by verification cost, not drafting quality.

Agentic frontier (*speculation*). Repetitive tool-call streams with $r \approx 0.85$ yield $\widehat{\alpha} \approx 0.70$; at K=16, Eq. 2 projects $\widehat{s} \approx 3.2\times$ on current hardware — but only if acceptance tracks redundancy as on S1–S2; S3 shows this assumption fails when model behavior shifts.

1–3 year horizon (*speculation*). As deployments move to 4B–70B models, our S3 data argue against porting small-model coefficients: re-fit $r \to \alpha$ per tier. Code-edit's S3 K=16 win (1.79$\times$) suggests larger models may unlock drafting on structured edits previously dominated by verify cost — a testable hypothesis, not a guarantee.

6. Limitations

(1) Five prompts per domain: headline cells carry bootstrap CIs, but several remain wide (S3 code-edit K=16 spans 0.32–1.91), and cell-level significance is not established everywhere. (2) Batch-1 greedy serving isolates mechanism but understates batching effects. (3) Six of 269 records

show a small negative no-proposal count (counter-snapshot timing); path A is unaffected. (4) Law transfer breaks at 4B (C18); extrapolation beyond measured tiers is unreliable. (5) γ is engine-specific (vLLM 0.25.0).

7. Conclusion

Context-sourced speculation is domain-conditional with order-of-magnitude spread, partially predictable by offline redundancy on small/mid models but *not* portable to 4B without re-measurement. Recipe: score traffic redundancy, apply Eq. 1–2 within the validated tier, and never enable blindly on math-like workloads.

Reproducibility

All measurements, harness code, prompt sets, the claims ledger mapping every number above to raw run IDs, and the autonomous-loop transcript are in the submission repository: https://github.com/Teora/ralphthon-icml-2026. Sweeps re-run via `experiments/driver.py --model <hf-id> --k <K>`; analysis via `experiments/analyze.py` and `experiments/redundancy.py`.

References

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. arXiv:2302.01318.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of LLM inference using lookahead decoding. In *ICML*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. SOSP 2023; vLLM.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

Oliaro, G., Jia, Z., Campos, D., and Qiao, A. SuffixDecoding: Extreme speculative decoding for emerging AI applications, 2024. arXiv:2411.04975.

Park, M. Suffix always-spec decoding for efficient Qwen3.5-4B inference on a single A10G, 2026. URL https://github.com/Teora/efficient-qwen-suffix-spec. Efficient Qwen Competition technical report (2nd of 43), AdaptFM @ ICML 2026.

Qwen Team. Qwen3 technical report, 2025. arXiv:2505.09388.

Saxena, A. Prompt lookup decoding, 2023. URL https://github.com/apoorvumang/prompt-lookup-decoding.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of ACL*, 2024.